

Bachelor INF
"Mobile Computing",
SS 2026
Prof. Dr. Natividad Martínez Madrid

- Project report -

PestaShare: A Hyper-Local Circular Economy App for Student Housing

Submitted by:
Mateo Benjamin Dillon Gangotena
Alexandros Bogiatzis

Contact address: mateo_benjamin.dillon@student.hs-reutlingen.de
alexandros.bogiatzis@student.hs-reutlingen.de

Submitted on: 24.06.2026

Abstract: *This seminar report presents PestaShare, a native Android application that establishes a functional circular economy within the Pestalozzistraße student dormitories in Reutlingen. Replacing fragmented WhatsApp coordination with a centralised bulletin board, the system supports listing, discovery and lifecycle tracking of household goods. The app is backed by Firebase: Cloud Firestore stores items, profiles and point histories and streams them to every device in real time, while Firebase Authentication restricts access to verified university accounts. A non-monetary Eco-Points token economy encourages active sharing.*

Table of contents

1. Introduction and Motivation.....	3
2. Goals of the Project.....	3
3. Requirements Analysis.....	3
3.1 Personas.....	4
3.2 Storyboard.....	4
3.3 User Stories.....	5
3.4 Requirement Specification.....	5
4. Conceptual Model of the Solution.....	6
4.1 Software Architecture.....	6
4.2 Navigation Model.....	7
4.3 Data Model.....	8
4.4 Activity Diagram — Requesting an Item.....	8
4.5 Sequence Diagram — Posting an Item.....	9
4.6 Mockups.....	10
5. Design Decisions.....	11
5.1 Native Kotlin / Jetpack Compose instead of React Native.....	11
5.2 Single Source of Truth via State Hoisting.....	11
5.3 Cloud Backend with Firebase Authentication and Cloud Firestore.....	11
5.4 The Eco-Point Incentive Loop.....	12
5.5 Explicit Item Lifecycle.....	12
5.6 Visual Identity and Component Choices.....	12
6. Results.....	12
6.1 Degree of Completion (Requirements Traceability).....	12
6.2 Testing and Evaluation.....	13
6.3 Known Limitations.....	14
7. Conclusion.....	14
8. Author Contributions.....	14
9. Annex: Use of Generative AI.....	15
9.1 Tools, Models and Contributions.....	15
9.2 Handling Hallucinations and Ensuring Quality.....	15
10. Declaration on oath.....	15

1. Introduction and Motivation

Student housing is, by its nature, a place of constant arrival and departure. At the Pestalozzistraße dormitories in Reutlingen, every semester brings a new cohort of residents and sees another wave of students move out — many of them international exchange students who can only take a suitcase home. The household items they leave behind, such as kettles, lamps, frying pans, tools and bed linens, are usually still perfectly usable, yet they routinely end up in the trash or abandoned in hallways, where they create clutter and even attract fines.

At the same time, newly arrived residents in the very same building spend money buying those identical items brand new. The two groups rarely connect. The coordination that does happen relies on large, noisy WhatsApp groups where a single “free toaster” message is buried within minutes, and it is impossible to track who is actually coming to collect what. The result is a paradox: usable goods are discarded on one floor while they are re-purchased on another.

PestaShare addresses this gap by turning the dormitory into a small, trusted circular economy. Instead of a linear “buy → use → discard” cycle, the app keeps resources circulating within the community: items are listed, lent, claimed and — only as a last resort — recycled. The motivation is both ecological (less waste, longer product life) and social (a more connected, mutually-supportive housing community). A lightweight, hyper-local “smart bulletin board” is the right tool for this, because the people sharing already live metres apart; the only thing missing is a structured, low-friction way to see what is available and to coordinate a pickup.

2. Goals of the Project

The overarching goal is to reduce avoidable waste in student housing by making local sharing easier than throwing things away. This breaks down into the following concrete objectives:

1. **Frictionless listing.** Let a departing resident photograph and publish an item in under a minute, directly from the phone.
2. **Discoverable inventory.** Provide a single, searchable and filterable feed of everything currently available in the building.
3. **Trusted exchange.** Restrict participation to verified dormitory residents and keep coordination inside the app, so users feel safe arranging a meetup.
4. **A circular incentive loop.** Use a non-monetary “Eco-Points” system that rewards giving and is spent on receiving, nudging the community toward active participation.
5. **A demonstrable, cloud-backed mobile prototype.** Deliver a fully navigable native Android application, synchronised across devices in real time, ready for a live demo.

3. Requirements Analysis

The requirements were derived in a user-centred way: we first defined personas representing the two sides of the exchange, sketched the experience as a storyboard, condensed the needs into user stories, and finally formalised them into a requirement specification.

3.1 Personas

Persona 1: Max, the Departing Exchange Student

Attribute	Description
Age / role	22 — international exchange student (ERASMUS)
Location	Pestalozzistraße 39, Room 512
Bio	Max is finishing a six-month exchange in Reutlingen and moves home next week. He has accumulated household items that will not fit in his suitcase.
Motivations	Avoid wasting good items; avoid a fine for leaving things in the hallway; help the next “generation” of students.
Pain points	Disorganised WhatsApp groups make it impossible to track who will actually pick up his toaster. He is busy packing and needs a fast way to list items.
Goals in PestaShare	List his kettle and bed linens, earn Eco-Points before leaving, and ensure items are recycled if unclaimed.

Persona 2: Lena, the Newly-Arrived Resident

Attribute	Description
Age / role	20 — first-semester Bachelor student, just moved in
Location	Pestalozzistraße 41, Room 108
Bio	Lena has just arrived with an empty room and a tight student budget. She needs basic household items but does not want to buy everything brand new.
Motivations	Save money, furnish her room quickly, and meet people in the building.
Pain points	She does not know anyone yet and has no overview of what neighbours are giving away.
Goals in PestaShare	Scroll an active feed, claim items she needs with Eco-Points, and safely arrange pickups with verified residents.

3.2 Storyboard



Figure 1 — Storyboard of the sharing journey.

3.3 User Stories

Role	Story
The Poster	As a departing exchange student, I want to quickly post my used household items so that I don't have to throw them in the trash when I move out and can contribute to the student community.
The Borrower	As a newly-arrived resident, I want to scroll through an active feed of available items so that I can save money instead of buying everything brand new.
Security	As a dorm resident, I want to know that my contact information is only revealed to verified students so that I feel safe arranging a meetup with another university student.

3.4 Requirement Specification

Functional Requirements

ID	Requirement	Priority
FR-1	Authenticate residents through a university-email login before any content is shown.	Must
FR-2	Display a feed of available items ("Discover" / Community Wall).	Must
FR-3	Search items by title.	Should
FR-4	Filter items by category (Electronics, Kitchen, Books, General).	Should
FR-5	Open an item detail view with description, pickup location, owner and point value.	Must
FR-6	Post a new item with title, category, location, description and photo.	Must
FR-7	Attach a photo from the device camera or gallery.	Must
FR-8	Request/claim an item, spending Eco-Points and updating its status.	Must
FR-9	Provide a pre-filled coordination message to the owner.	Should
FR-10	Edit a previously posted item.	Should
FR-11	Delete a posted item.	Should
FR-12	Mark an owned item as "Given" once handed over.	Must
FR-13	Show the user's live Eco-Point balance.	Must
FR-14	Redeem Eco-Points for campus rewards in an "Eco Market".	Could
FR-15	Keep a point-transaction history log.	Should
FR-16	Surface coordination messages in an activity/inbox view.	Should
FR-17	Reflect each item's lifecycle state through status badges.	Must
FR-18	Offer a profile with statistics and sign-out.	Should
FR-19	Flag long-idle items with a recycling suggestion.	Could

Non-Functional Requirements

ID	Requirement
NFR-1	Native Android application (minSdk 26 / targetSdk 35) built with Jetpack Compose and Material 3.
NFR-2	Responsive, edge-to-edge UI that follows a consistent visual identity (green = sustainability, blue = action).
NFR-3	Data is persisted in a cloud backend (Cloud Firestore) and survives an app restart, a sign-out and a reinstall.
NFR-4	The interface must update reactively the moment underlying data changes — across devices — with no manual refresh.
NFR-5	Privacy: owner contact/pickup details are only shown to authenticated, domain-verified residents inside the app.
NFR-6	Lists must scroll smoothly using lazy/virtualised rendering.

4. Conceptual Model of the Solution

4.1 Software Architecture

PestaShare follows the modern Android single-activity architecture. A single MainActivity hosts the entire Compose UI tree. All seven screens are stateless @Composable functions that receive their data as parameters and report user actions back through callbacks. The application state — login status, the Eco-Point balance, and the lists of items, chats and history — is hoisted into a single NavGraph() composable, which acts as the single source of truth and contains the NavHost and bottom navigation bar. State flows down into the screens and events flow up into the state holder. Crucially, that hoisted state is no longer seeded from in-memory lists: it is fed by live Cloud Firestore snapshot listeners, so the UI reflects the cloud the moment data changes.

PestaShare — Software Architecture

Single-Activity · Jetpack Compose · real-time Cloud Firestore

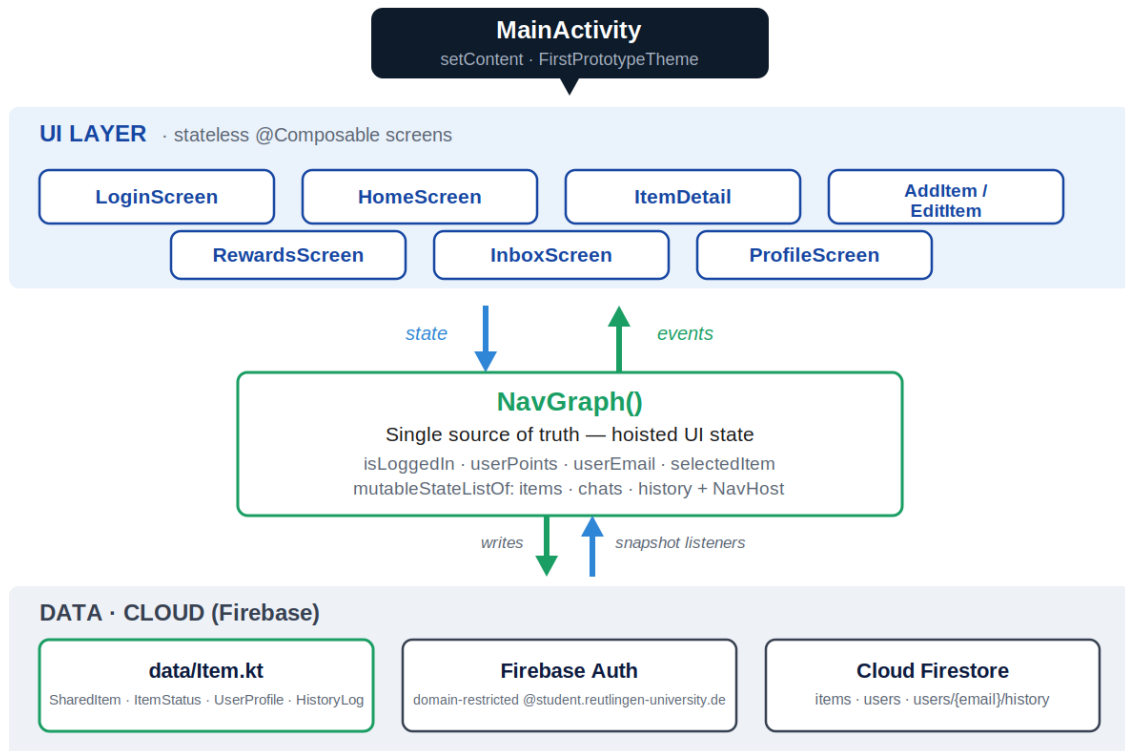


Figure 2 — Layered software architecture and unidirectional state flow.

4.2 Navigation Model

After the authentication gate, five primary destinations are reachable through the bottom navigation bar (Home, Add, Market, Activity, Profile). Two secondary screens, “Item Detail” and “Edit Item”, are pushed onto the back stack on top of the primary destinations and pop back when the user finishes. The authentication state itself is persistent: `NavGraph()` reads `FirebaseAuth.currentUser`, so a returning user with an active session bypasses the login screen entirely. Navigation uses `launchSingleTop` with state saving/restoring so tab switches feel instant and preserve scroll position.

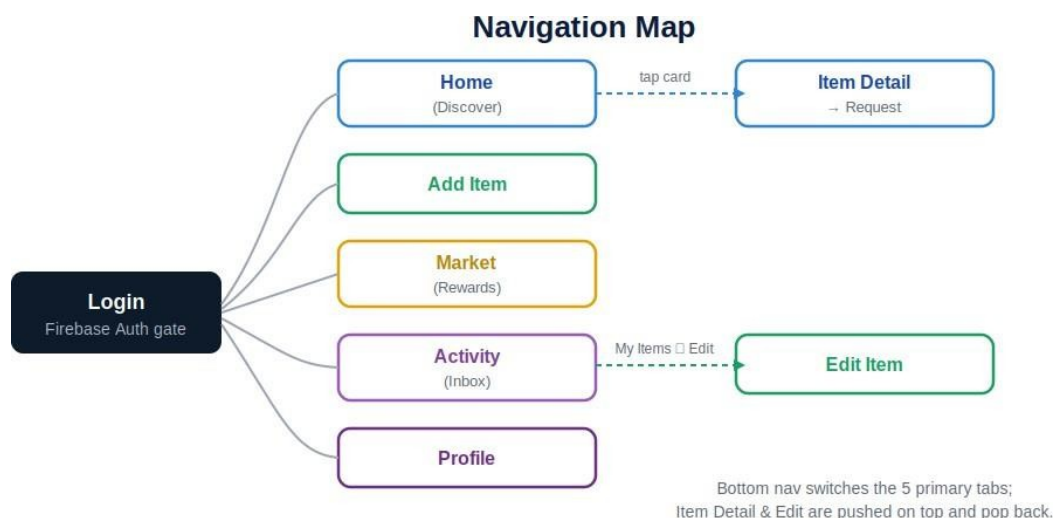
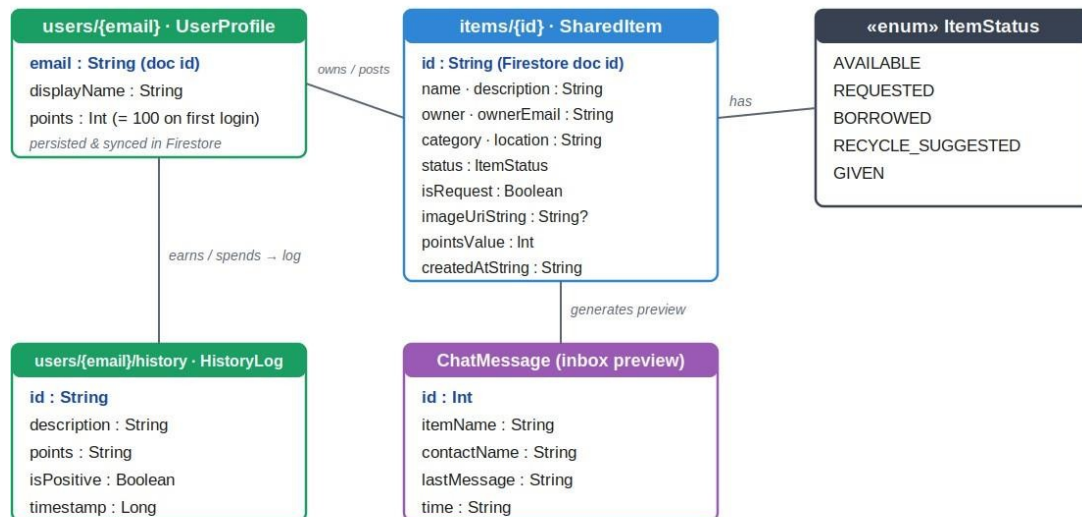


Figure 3 — Screen navigation map.

4.3 Data Model

The domain centres on the `SharedItem` entity and its `ItemStatus` lifecycle enum. Supporting structures model the user session (`UserProfile`, derived from the authenticated email plus a persisted point balance), the redeemable reward options, the coordination `ChatMessages`, and the `HistoryLog` of point transactions. These objects live in Cloud Firestore rather than in memory: a global items collection stores all shared resources, and each `users/{email}` document holds a profile plus a history sub-collection of point transactions. Only the lightweight inbox chat previews remain in reactive in-memory state.

Conceptual Data Model — Cloud Firestore



Doc keys shown in blue - items & users (+ history) are real-time Firestore collections; chat previews remain in-memory

Figure 4 — Conceptual data model (Firestore collections and entities).

4.4 Activity Diagram — Requesting an Item

The most important transaction is claiming an item. The activity diagram below shows the guard conditions (availability and sufficient points) and the resulting Firestore writes — deducting points on the user document, updating the item status to `REQUESTED`, adding a history-log entry and building the coordination message. Because each write commits in the cloud, two users cannot claim the same item simultaneously, and the snapshot listeners then propagate the new balance and badges to every device.

Activity — Request an Item

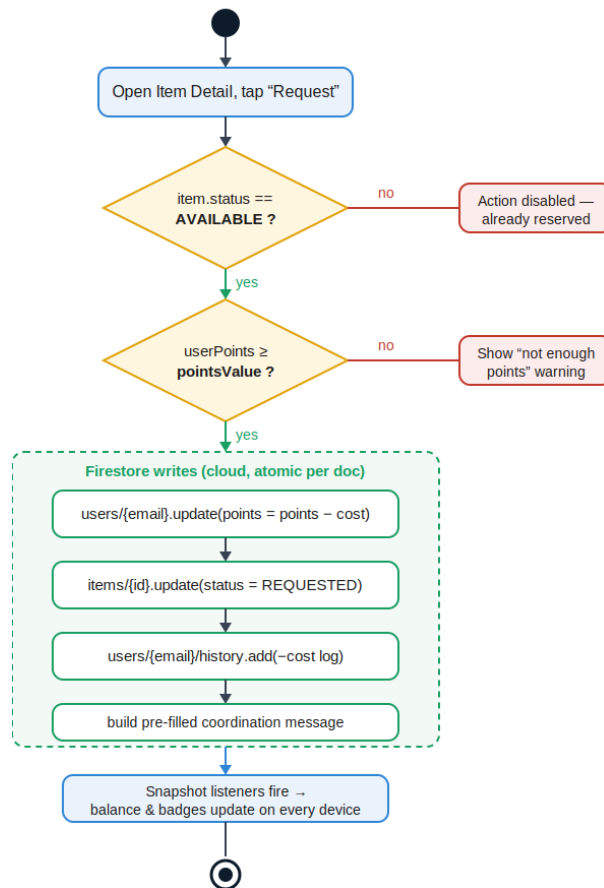


Figure 5 — Activity diagram for the “request an item” flow (Scenario 1).

4.5 Sequence Diagram — Posting an Item

Posting an item involves the user, the Add-Item screen, the system photo picker, the central state holder and Cloud Firestore. The sequence diagram traces a successful post, including image selection, client-side validation, the write to the items collection, and the reactive re-render of the home feed driven by the snapshot listener rather than a manual refresh.

Sequence — Post an Item

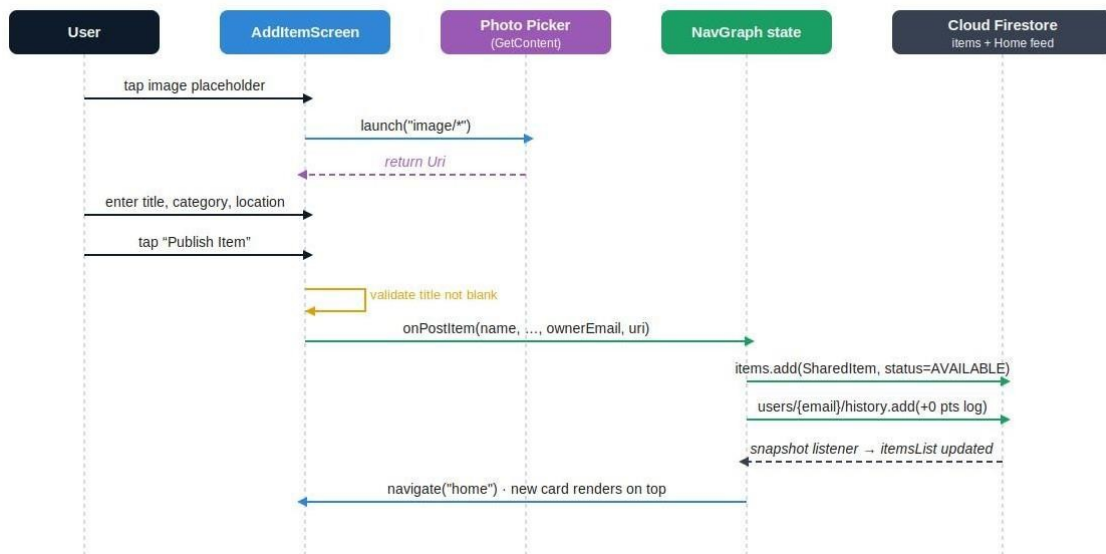
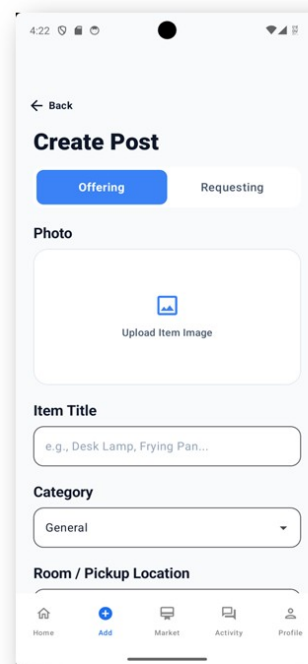
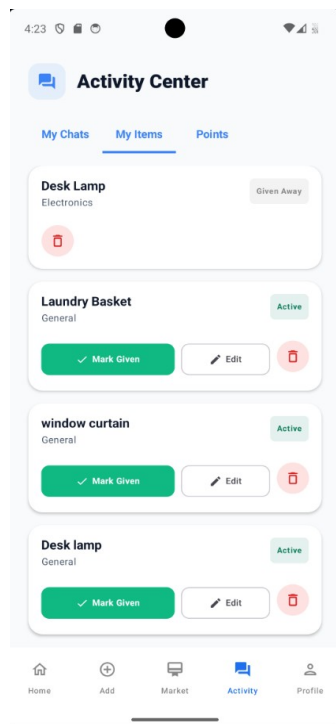
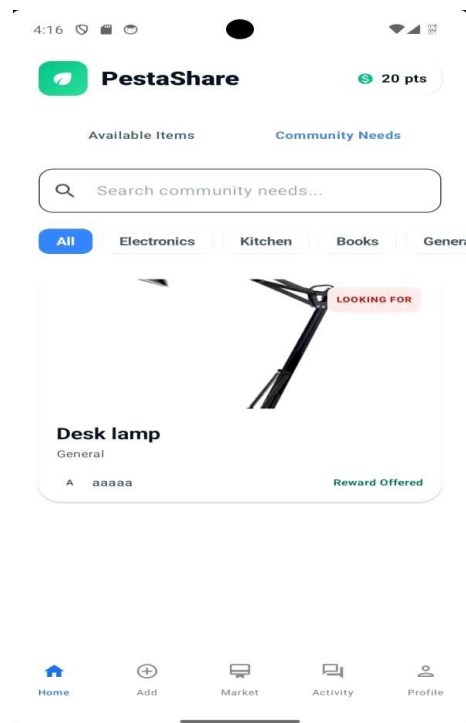
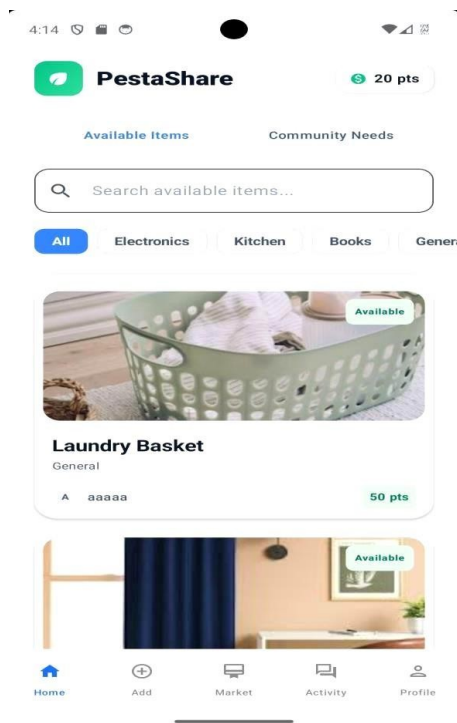
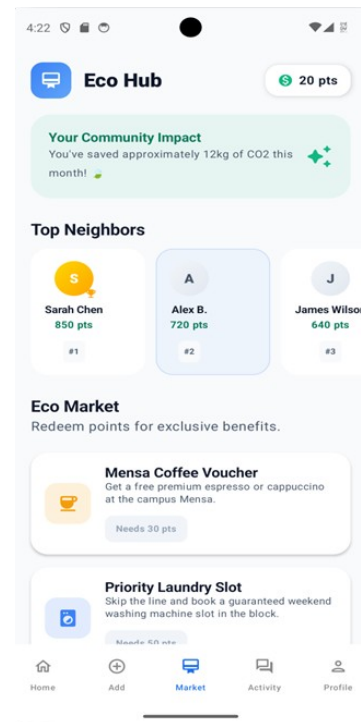
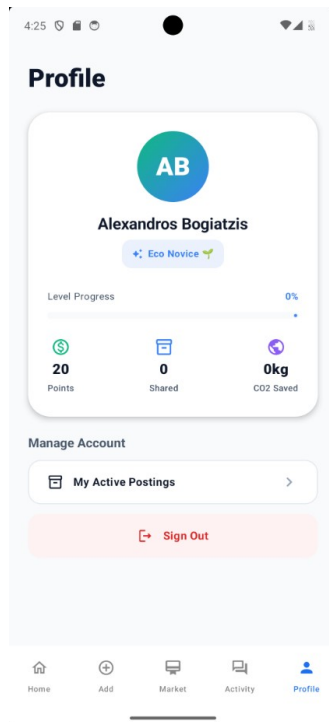


Figure 6 — Sequence diagram for the “post an item” flow (Scenario 2).

4.6 Mockups





5. Design Decisions

5.1 Native Kotlin / Jetpack Compose instead of React Native

The initial proposal envisioned React Native. During implementation we pivoted to native Android with Kotlin and Jetpack Compose. The deciding factors were: a declarative, state-driven UI model that maps cleanly onto our hoisted-state architecture; first-class access to platform features such as the photo picker; Material 3 components out of the box; and the fact that the Mobile Computing context made native Android the more instructive target. The few React Native artefacts that remain in the repository are vestiges of the original plan and are not part of the shipped app.

5.2 Single Source of Truth via State Hoisting

Rather than scattering state across screens or introducing a heavier architecture, all shared state is hoisted into `NavGraph()` using `remember { mutableStateOf(...) }` and `mutableStateListOf(...)`. Screens are pure functions of their inputs. The hoisted lists are kept in sync with Cloud Firestore through `addSnapshotListener` callbacks, so any change — a claim, a redemption, a new post, whether made on this device or another — flows into the snapshot state and propagates to every affected screen automatically, satisfying NFR-4 with almost no boilerplate.

5.3 Cloud Backend with Firebase Authentication and Cloud Firestore

The original prototype kept all data in memory and persisted only the point balance with `SharedPreferences`. To turn PestaShare into a genuinely collaborative, multi-device platform we migrated to Firebase. Two services do the work:

- **Firebase Authentication** acts as a secure entry gate. Account creation and sign-in are restricted to addresses ending in `@student.reutlingen-university.de`, keeping the community private, and the session is persistent so users are not asked to log in again on every launch.

- **Cloud Firestore** replaces the in-memory store. A global items collection holds all shared resources; a users collection holds each profile, with a history sub-collection for point transactions. The app attaches snapshot listeners instead of fetching manually, so the Home feed, the point balance and the activity log update the instant the underlying documents change.

Point changes (requesting an item, redeeming a reward) are written as targeted updates to the user document, and item state transitions are written directly to the item document, which keeps the lifecycle consistent in the cloud and prevents two users from claiming the same item at once.

5.4 The Eco-Point Incentive Loop

A non-monetary point currency was chosen over real money to keep the exchange community-spirited and low-stakes. New users start with a balance of 100, stored in their Firestore profile. Requesting an item spends points; redeeming a campus reward spends points; every action is written to a transparent history log in the user's history sub-collection. This creates a gentle circular incentive: to keep receiving, residents are encouraged to keep giving. Points are intentionally lightweight rather than a strict ledger, matching the project's goals.

5.5 Explicit Item Lifecycle

Item state is modelled as an explicit enum — AVAILABLE, REQUESTED, BORROWED, RECYCLE_SUGGESTED, GIVEN — rather than loose boolean flags. This makes the UI logic declarative (each state maps to a coloured badge and a set of permitted actions) and makes the lifecycle easy to reason about and extend. Because the status lives on the Firestore item document, every device sees the same lifecycle state at the same time.

5.6 Visual Identity and Component Choices

The design language pairs emerald green (sustainability, balances) with an electric blue (primary actions) over a clean slate-on-white surface. We used Material 3 throughout, Coil for asynchronous image loading, lazy lists for all feeds, and a bottom navigation bar as the primary structure.

6. Results

The project delivered a complete, navigable, cloud-backed native Android application that demonstrates the circular-economy concept end-to-end. All primary flows — login, browse, search/filter, post, request, manage and redeem — are functional in the running app and synchronise live across devices.

6.1 Degree of Completion (Requirements Traceability)

Each functional requirement was traced back to the implementation. “Implemented” means fully working in the app; “Partial / Mocked” means the behaviour is present but simplified for the project scope.

ID	Requirement (short)	Status
FR-1	University-email login gate (Firebase Auth, domain-restricted)	Implemented
FR-2	Discover feed	Implemented
FR-3	Search by title	Implemented

ID	Requirement (short)	Status
FR-4	Category filter	Implemented
FR-5	Item detail view	Implemented
FR-6	Post item (full form)	Implemented
FR-7	Attach photo (gallery/camera)	Implemented
FR-8	Request/claim item with points	Implemented
FR-9	Pre-filled coordination message	Implemented
FR-10	Edit item	Implemented
FR-11	Delete item	Implemented
FR-12	Mark as Given	Implemented
FR-13	Live point balance	Implemented
FR-14	Eco Market redemption	Implemented
FR-15	Point history log	Implemented
FR-16	Activity / inbox view	Implemented
FR-17	Status badges / lifecycle	Implemented
FR-18	Profile + sign-out	Implemented
FR-19	Recycle suggestion for idle items	Partial / Mocked

Summary: 18 of 19 functional requirements are fully implemented. The remaining one, FR-19, is present but simplified: an item's recycle status is set on seed data rather than computed automatically from its age. FR-1 is now fully met — Firebase Authentication enforces a real, domain-restricted login rather than accepting any credentials.

6.2 Testing and Evaluation

Quality was assured through a combination of methods appropriate to the project scope:

- **Manual functional testing** on an emulator and a physical device. Each user story was walked through end-to-end (post → appears on the feed; request → points deducted and status changed; redeem → balance updates; edit/delete/mark-given → list updates).
- **Multi-device synchronisation testing.** With two clients signed in, a change made on one device (a new post or a claim) was observed appearing on the other in real time via the Firestore listeners.
- **Authentication testing.** Sign-in and registration were verified to reject non-university domains and empty fields, and to surface Firebase error messages.
- **Scenario-based testing.** The two formal scenarios (claiming and posting) were tested including their error paths: insufficient points blocks the request and shows a warning; an already-reserved item disables its action button; an empty title blocks publishing.
- **Compile-and-run verification of AI-assisted code.** Every generated Kotlin unit was compiled and executed; non-compiling or deprecated API usage was corrected before being accepted.

- **Heuristic UI review.** Screens were reviewed against basic usability heuristics — visible system status (badges, balances, loading feedback), error prevention (form validation), and consistency (a single visual language).

6.3 Known Limitations

- Images are still referenced by device-local URIs rather than uploaded to Cloud Storage, so a photo is only visible on the device that posted it; hosted image storage is the next step.
- Authentication restricts access by email domain but does not yet add an email-link verification step on top of the password credential.
- The “Rescued” profile statistic is a placeholder and the recycle suggestion is not yet derived from item age.
- Coordination is represented through message previews rather than a full real-time chat.

7. Conclusion

PestaShare shows that a focused, hyper-local sharing app can make reuse the path of least resistance for students. The application delivers the full intended experience — listing, discovering, claiming and managing items, backed by an Eco-Point incentive loop and a clear item lifecycle — inside a clean, native Android app that now synchronises in real time across devices through Cloud Firestore and gates access through Firebase Authentication.

With the shared backend and verified login already in place, the natural next steps move the app toward a fully deployable product: Cloud Storage to host item images instead of device-local URIs, push notifications when an item is claimed, an email-link verification step layered on the domain-restricted login, an age-based recycle suggestion computed from createdAt, and a real-time chat to replace the message previews. None of these require changing the conceptual model — the app was built so that they can be layered on top of the existing Firestore structure.

8. Author Contributions

This section records which team member was responsible for which parts of the work and where the two collaborated, so that the individual contributions can be told apart. The division reflects how the codebase actually evolved in the shared repository.

Shared participation: Both members participated together in shaping the overall concept and architecture, integrating each other’s features, resolving merge conflicts on the main branch, and carrying out the testing and evaluation described in Section 6.2. The requirements analysis, the design decisions and the assembly of this report were likewise a shared effort, with each member reviewing and verifying the other’s contributions.

Team member	Tasks led / responsible for
Mateo Benjamin Dillon Gangotena	Architected the core application skeleton and engineered the comprehensive single-activity navigation framework using Jetpack Compose. Wired MainActivity directly to NavGraph() to secure a fluid, unidirectional state flow across all functional screen scaffolds. Pioneered the premium UI visual overhaul, state theme bindings, and developed the mathematical execution logic for the Eco-Point token calculation system.
Alexandros Bogiatzis	Created the app logo, designed the app Add-Item screen mockup and refined

Team member	Tasks led / responsible for
	the interfaces of the other screens. Implemented the university-email login flow and the image-based item posting. Built the “My Items” tab and the location-tag feature for items, and added explanatory comments throughout the codebase. Led the Firebase migration: Authentication with university-mail verification and the Cloud Firestore database setup for posts. Also handled branch merging to keep the shared repository in sync.

9. Annex: Use of Generative AI

Generative AI was used as an assistant throughout the project. The team retained full responsibility for design decisions, integration and verification.

9.1 Tools, Models and Contributions

Tool / Model	Used for
ChatGPT	Idea refinement, persona and user-story wording, scenario descriptions, parts of this documentation.
Claude	Kotlin & Jetpack Compose code generation, the Firebase Authentication and Cloud Firestore integration, refactoring, debugging, and assembling this report and the diagrams.
Gemini	Storyboard comic panels and the app logo concept, and help fixing errors in Android Studio.

9.2 Handling Hallucinations and Ensuring Quality

Because generative models can produce confident but incorrect output, we applied several safeguards:

- All AI output (text, code, images) was reviewed and edited by the team; the model was treated as a drafting aid, not an authority.
- **Iterative prompting:** when an answer was wrong or unsatisfactory, we narrowed the prompt with concrete constraints (exact versions, the surrounding code, the precise error) instead of accepting the first response.
- We rejected suggestions that pulled the project beyond the agreed scope, keeping the result coherent and demonstrable within the timeline given.

10. Declaration on oath

We hereby declare that we have written the present work on our own, that we have not used any external help other than declared and that all helping material and sources used for the work have been properly named and referenced. We are aware that an untruthful declaration will be considered as a deceit.

Reutlingen 23/06/2026
Place, Date

ALEXANDROS BOGIATZIS
Signature (First and last name)

Reutlingen 23/06/2026
Place, Date

Mateo Benjamin Dillon Gangotena
Signature (First and last name)

Github: <https://github.com/BenjaminDillon2003/PestaShare.git>